



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Intro to Synchronous Digital Systems

Great Idea #1: Abstraction (Levels of Representation/Interpretation)

How do we design the hardware needed to execute machine code?

High Level Language Program (e.g., C)

Compiler

Assembly Language Program (e.g., RISC-V)

Assembler

Machine Language Program (RISC-V)

```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

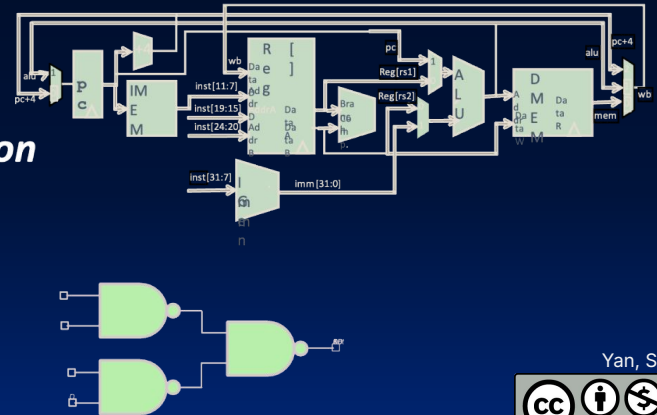
```
lw      x3, 0(x10)
lw      x4, 4(x10)
sw      x4, 0(x10)
sw      x3, 4(x10)
```

1000	1101	1110	0010	0000	0000	0000	0000
1000	1110	0001	0000	0000	0000	0000	0100
1010	1110	0001	0010	0000	0000	0000	0000
1010	1101	1110	0010	0000	0000	0000	0100

Hardware Architecture Description (e.g., block diagrams)

Architecture Implementation

Logic Circuit Description (Circuit Schematic Diagrams)





Why Study Hardware Design?

- Why study **hardware design**? Go beyond "just programming."
 - Why is my computer so slow? Why does my battery run down?
- To really understand how computers work, we need to understand the complete stack, including the physical level.
 - Understand capabilities and limitations of HW in general,
 - and processors in particular.
 - ...and when we need to choose custom hardware!

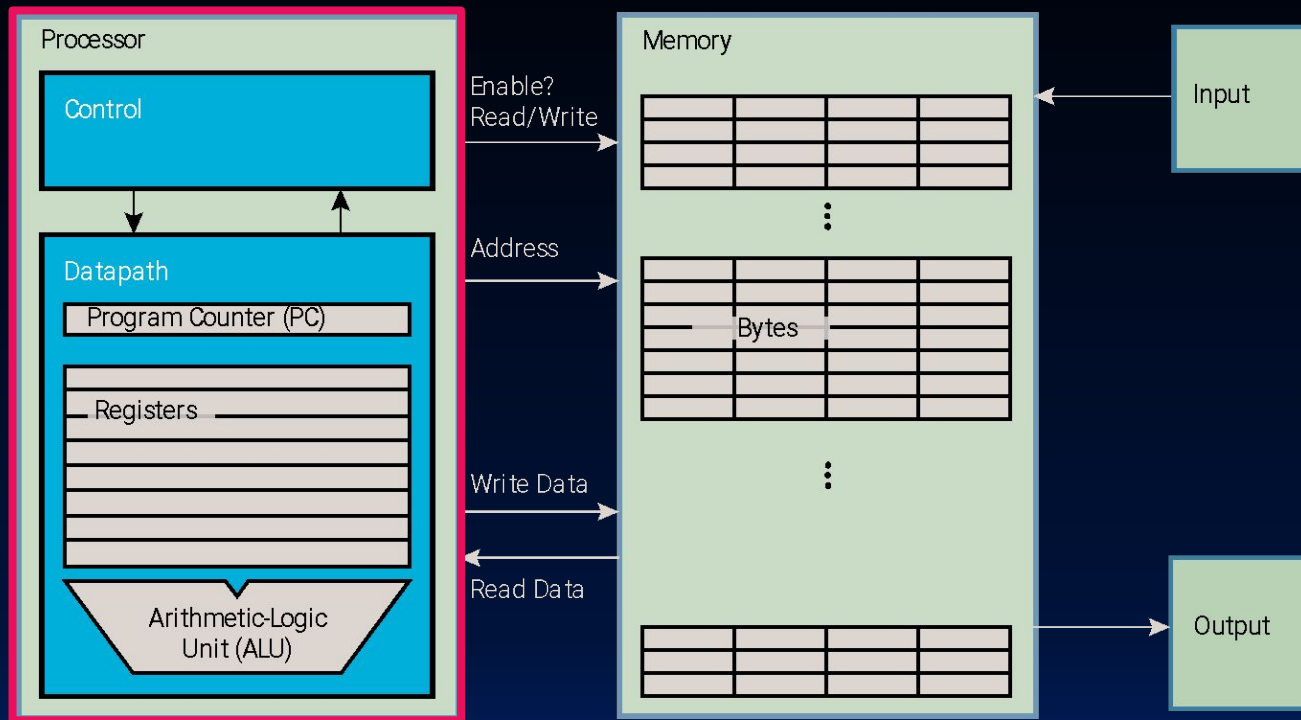
The principles we teach now will likely still apply in 30 years, even if/when base technology changes!



How do we build a Single-Core Processor?

Processor (CPU):
the active part of the computer that does all the work (data manipulation and decision-making).

To build a CPU, we need to understand **synchronous digital systems**.

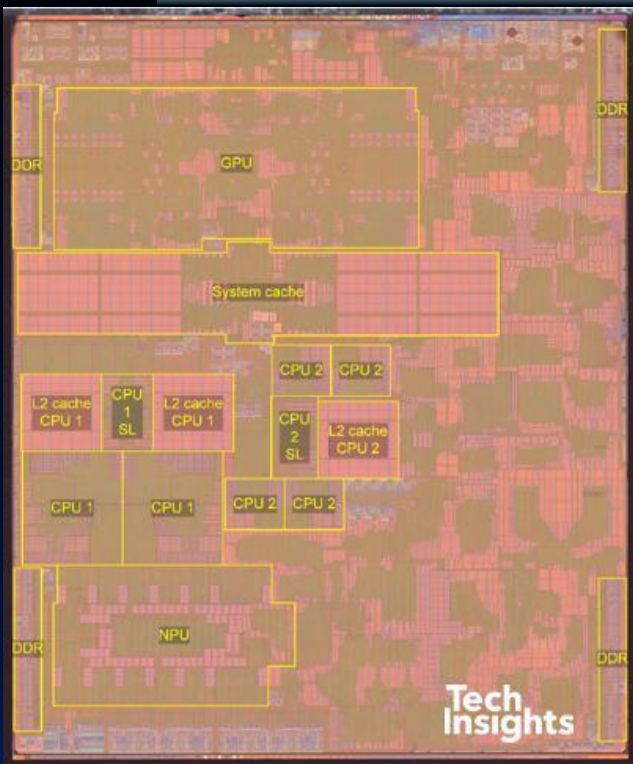




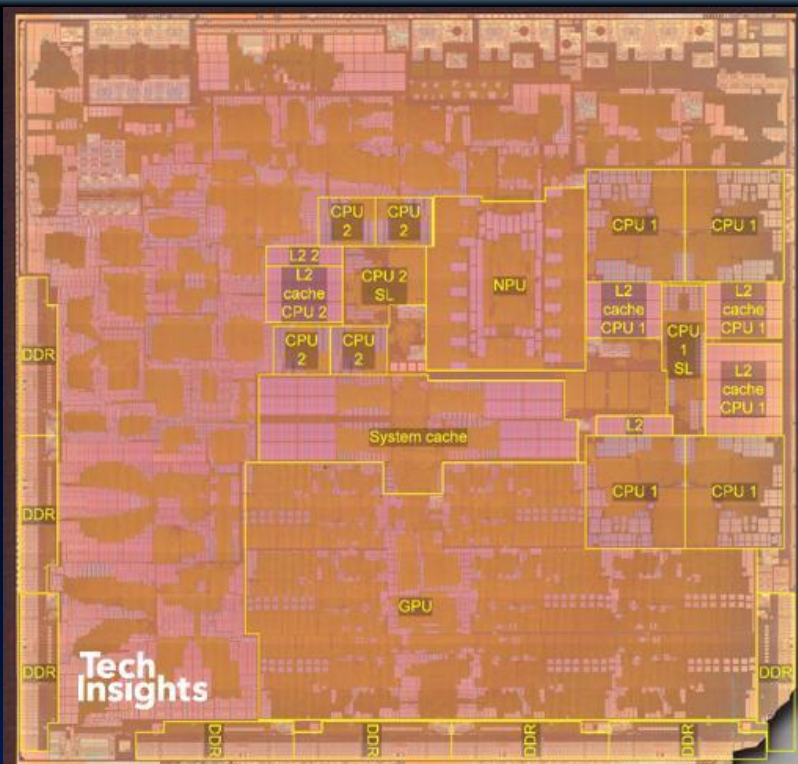
Moore's Law

- Synchronous Digital Systems
- Moore's Law
- Logic Gates and Truth Tables

Modern IC (Integrated Chip) Photos



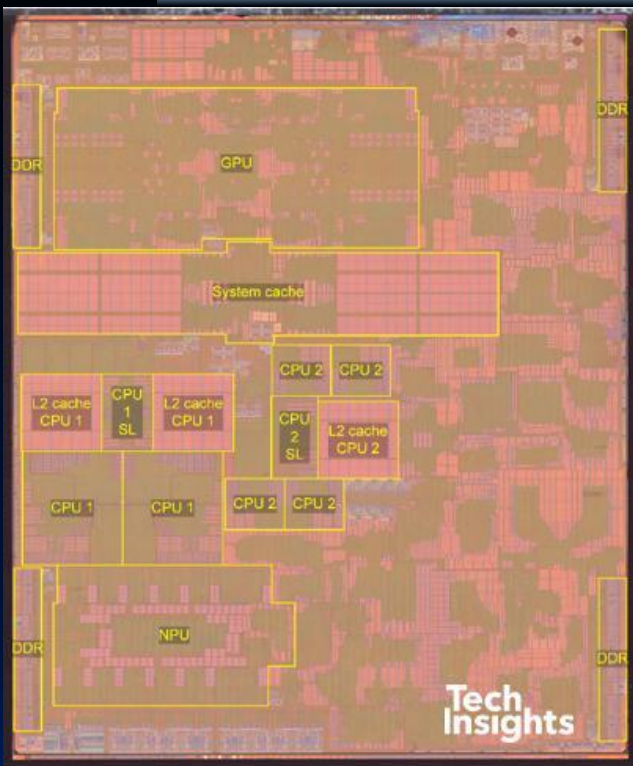
Apple A14 Bionic (2020)



Apple M1 Chip (2020)

On chip, all circuits are made from **wires and transistors.**

Modern ICs have Multi-core/Domain-Specific Processors



Apple A14 Bionic

Apple Implementation of ARM v8.5-a:

- 11.8 billion transistors!
- 5W power consumption
- **Six cores** up to 3.0 GHz:
 - Two high-performance, Four energy-efficient
- CPU supports 64-bit data
- GPU for working with graphical data
- NPU "Neural Engine" dedicated neural net HW
- AMX matrix scalar multiplication accelerators
- L2 cache: 4-8MB

Transistors: History

- Transistors: semiconductor device to amplify or **switch** signals.
 - Key component in ALL modern electronics
 - John Bardeen, William Shockley, Walter Brattain
- Before Transistors:
 - Vacuum Tubes



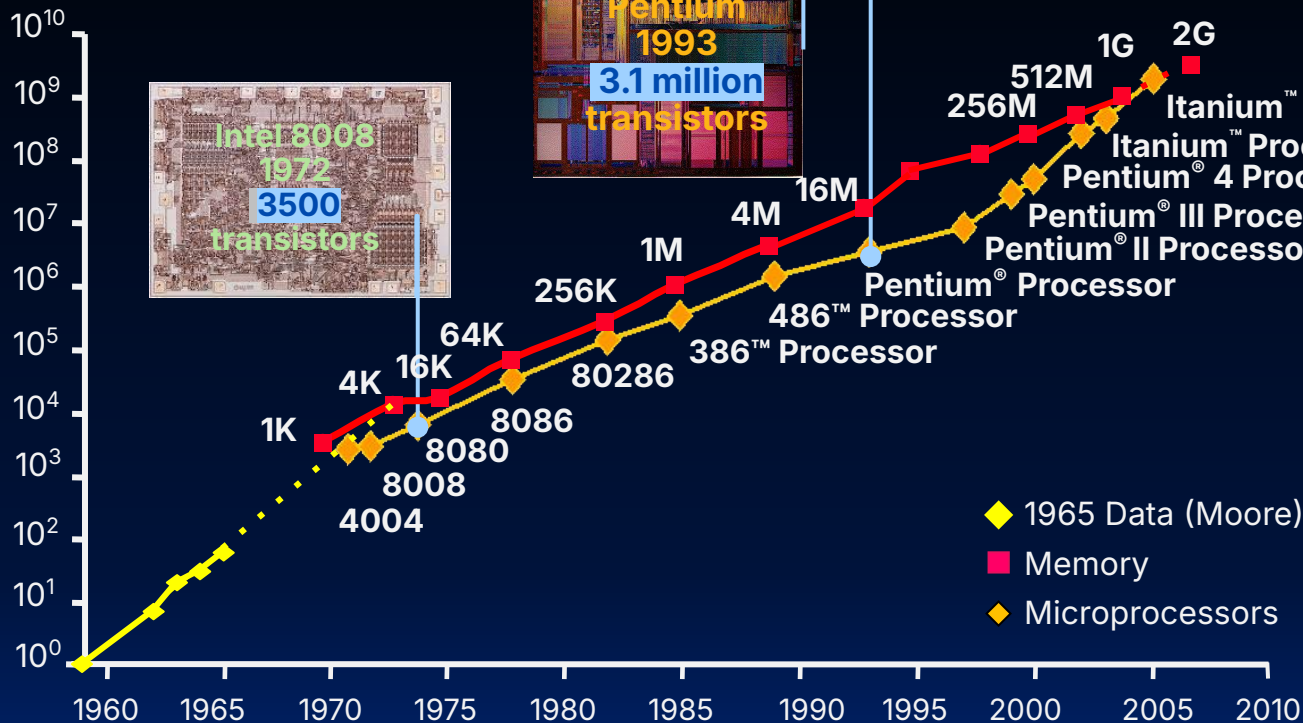
- After Transistors:
 - **Integrated circuit**,
 - microprocessor



"The Transistor was probably THE most important invention of the 20th Century"
 - Ira Flatow, Transistorized! (PBS Special)

Great Idea #2: Moore's Law - 2005

Transistors
Per Die

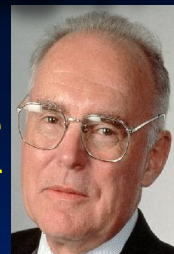


“Reduced cost is one of the big attractions of integrated electronics, and the cost

advantage continues to increase as the technology evolves toward the production of larger and larger circuit functions on a single semiconductor substrate.”

Electronics, Volume 38,
Number 8, April 19, 1965

Gordon Moore
Intel Cofounder
B.S. Cal 1950!



Yan, SP26

Moore's Law...?

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count

50,000,000,000

10,000,000,000

5,000,000,000

1,000,000,000

500,000,000

100,000,000

50,000,000

10,000,000

5,000,000

1,000,000

500,000

100,000

50,000

10,000

5,000

1,000

Year in which the microchip was first introduced

Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Seems to be tapering?



Synchronous Digital Systems

- Synchronous Digital Systems
- Moore's Law
- Logic Gates and Truth Tables



Synchronous Digital System (SDS)

The hardware underlying almost every processor is a **Synchronous Digital System**.

- **Synchronous**: All operations coordinated by a central **clock**.
 - "Heartbeat" of the system!
- **Digital**: Represent all values by discrete values, i.e., **binary digits**.
 - We've seen how to represent many symbols via 1s and 0s.
This representation extends to electrical signals!



SDS: Two Types of Circuits

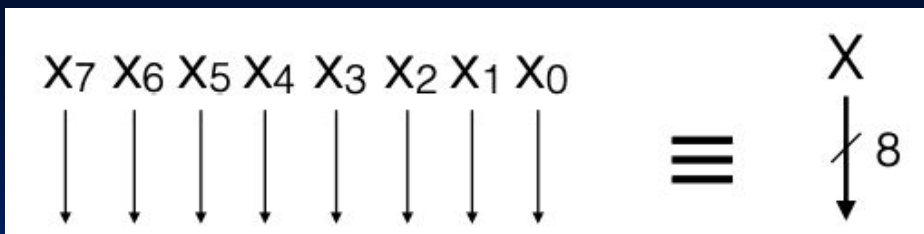
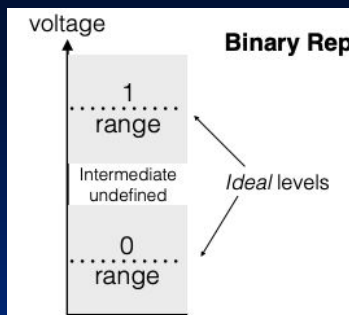
Synchronous Digital Systems consist of two basic types of circuits:

1. Combinational Logic circuits (today and SDS II)
 - Output is a function of the inputs only.
 - Similar to a pure function in mathematics, $y = f(x)$.
 - No way to store information from one invocation to the next, no side effects.
2. State Elements (in SDS III)
 - Circuits that store information.
 - Example: Registers

Our Goal: Implement a **RISC-V processor**.
This SDS can execute RISC-V instructions.

Wires: Binary Representation of Signals

- On a chip/PC board, **wires** (i.e., electrical nodes) provide electrical signals and are used to represent **variables**.
 - A wire can take on different values at different points in time.
- Binary signal representation**. Use voltage levels to signal 0 or 1.
- Instead of making **signals** complex, we keep it simple and push complexity later into how we **combine** signals.
 - (below) **Bundles of wires represent multi-bit variables**.





Logic Gates and Truth Tables

- Synchronous Digital Systems
- Moore's Law
- Logic Gates and Truth Tables



Combinational Logic Circuits

- To design circuits that perform complex operations on binary signals:
 - First, define **primitive operators**.
 - Then, compose these primitive operators to build complex operations.
- Primitive operators for combinational logic are called **logic gates**.
 - The simplest logic gates are unary/binary operators that take as input **one/two binary variables** and output **one binary value**.

Example Logic Gate: AND

- Ex: 2-input AND logic gate.
 - Two binary inputs a, b
 - Binary output y

AND

$y = \text{AND}(a, b)$
 $= 1$ iff both
 a, b are 1
 (0 otherwise)

Function Definition

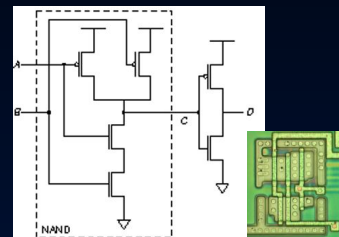
a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

Truth Table

Enumerate each
 input combination
 and the
 corresponding
 output value



Symbol for a
 2-input **AND** gate



CMOS transistor
 circuit for AND
 logic gate
 (out of scope, [source](#))

2-Input logic Gates: AND, OR, NOT

AND



a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

OR



a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

NOT



a	y
0	1
1	0

NAND

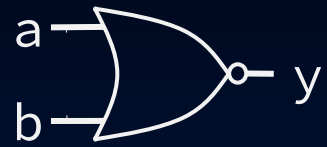
("not AND")



a	b	y
0	0	1
0	1	1
1	0	1
1	1	0

NOR

("not OR")



a	b	y
0	0	1
0	1	0
1	0	0
1	1	0

XOR

("exclusive OR")



a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

More 2-Input Logic Gates: NAND, NOR, XOR

NAND

("not AND")



a	b	y
0	0	1
0	1	1
1	0	1
1	1	0

NOR

("not OR")



a	b	y
0	0	1
0	1	0
1	0	0
1	1	0

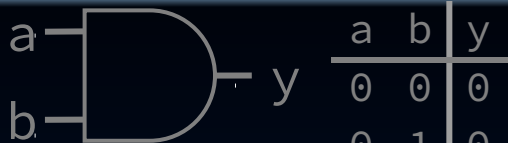
XOR

("exclusive OR")



a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

AND



a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

OR



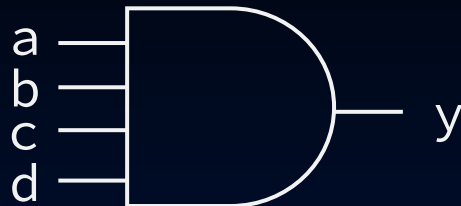
a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

NOT



a	y
0	1
1	0

- Gates all extend to n-input definitions:



$y = \text{AND}(a, b, c, d)$
 $= 1$ iff all of
 a, b, c, d are 1
 $(0$ otherwise)

If 2-input XOR is really $\neq$, what is an N-input XOR?

1 when they are all different

0%

1 when they are all the same

0%

0 when they are all different

0%

0 when they are all the same

0%

None of the above

0%



26

Generalizing to N-Input Logic Gates

An N-input NAND is NOT of an (N-input AND)

An N-input NOR is NOT of an (N-input OR)

An N-input XOR is 1 if the number of 1s on all the inputs is odd

NAND

$$y = \text{NAND}(a, b)$$

$$= 1 \text{ iff}$$

$$\text{AND}(a, b) = 0$$



a	b	y
0	0	1
0	1	1
1	0	1
1	1	0

NOR

$$y = \text{NOR}(a, b)$$

$$= 1 \text{ iff}$$

$$\text{OR}(a, b) = 0$$



a	b	y
0	0	1
0	1	0
1	0	0
1	1	0

XOR

("exclusive OR")



a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

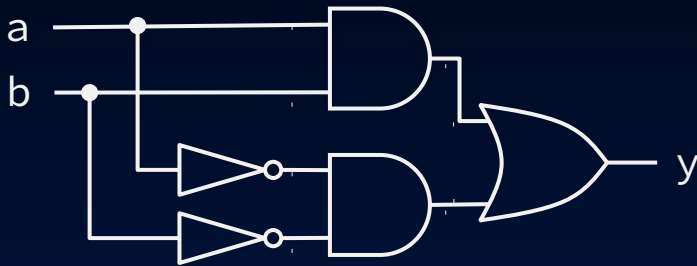


Circuit Design: Compose Blocks

- Synchronous Digital Systems
- Moore's Law
- Logic Gates and Truth Tables

Circuit → Truth Table (1/3)

- Logic gates are basic building blocks to design circuits (gate diagrams) that perform meaningful functions.
 - Particularly useful:** AND, OR, NOT
 - (also, Universal gates: NAND, NOR 🏠)



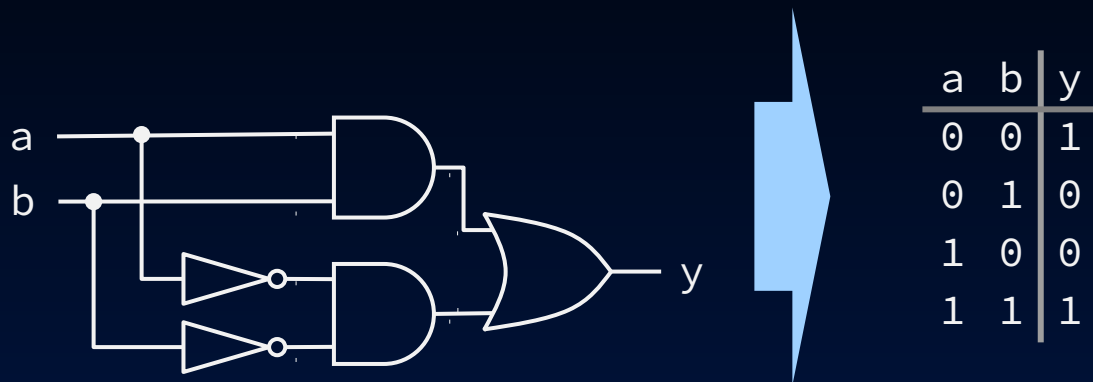
What does this circuit do?

Step 1: What is its truth table?



Circuit → Truth Table (2/3)

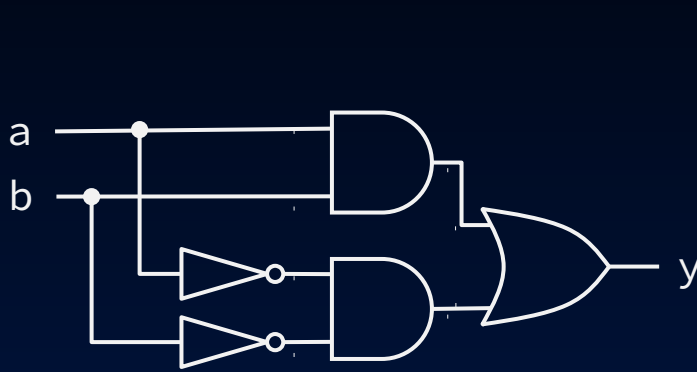
- Compare circuit!



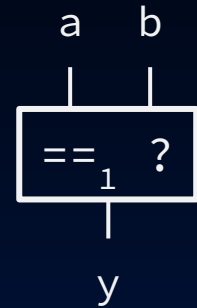
Implement with:

- Our bitwise compare block
- More logic gates

- Compare circuit!



a	b	y
0	0	1
0	1	0
1	0	0
1	1	1



$y = 1$ iff a, b equal

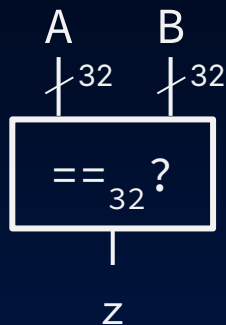
After we have designed this circuit with gates...

We can represent it as a **combinational logic block** to compose more complex circuits.



32-bit Equality Compare Circuit (1/3)

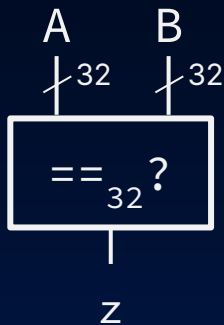
- Example: RV32I instruction `beq rs1, rs2, label`
 - If value in **rs1 == value** in rs2, then go to instruction at label, else go to next instruction.
- Somewhere in the processor must be a **circuit that compares 32-bit values**.



$z = 1$ iff $A == B$,
32-bit A, B; 1-bit z

32-bit Equality Compare Circuit (2/3)

- The 32-bit compare block has a truth table that is prohibitively expensive to write out by hand.



$z = 1$ iff $A == B$,
32-bit A, B; 1-bit z

a_{31}	...	a_1	a_0	b_{31}	...	b_1	b_0	z
0	...	0	0	0	...	0	0	1
0	...	0	0	0	...	0	1	0
	...				...			...
1	...	1	1	1	...	1	1	1

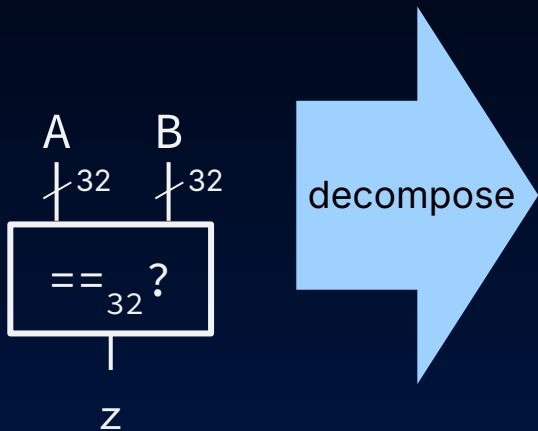
How many rows?

$$2^{32+32} = 2^{64} \text{ rows}$$

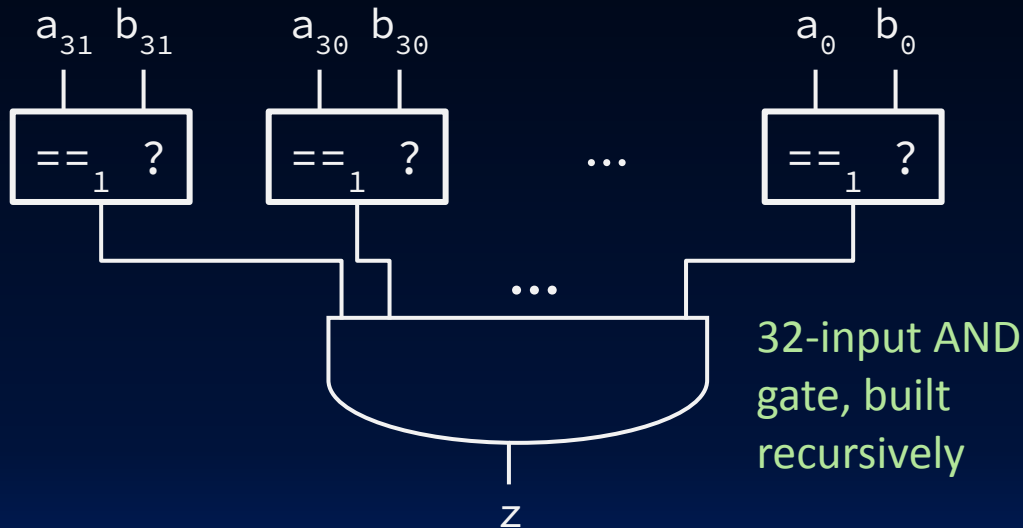
32-bit Equality Compare Circuit (3/3)

- However, we know the target functionality and can **decompose the logic** into smaller blocks.

Modular design: If truth tables are too big to construct, define smaller blocks first.



$z = 1$ iff $A == B$,
32-bit A, B ; 1-bit z



$z = 1$ iff $a_{31} == b_{31}$ AND $a_{30} == b_{30}$ AND ... AND $a_0 == b_0$

Yan, SP26

32-input AND gate?

Functionally, the 32-input AND gate can be built recursively:

